Enhanced SDN Controller Placement and Dynamic Load Balancing for Campus Network Optimization

Using K-Median Clustering and Dynamic Weighted Round Robin (DWRR)

Dishitha Pakala
Department of Computer Science, Kennesaw State University
Computer Networks Architecture — Spring 2026

Abstract— Large campus networks experience high control-plane latency, poor load distribution, and misplaced SDN controllers under default configurations. This paper proposes a dual-layer optimization framework: K-Median Clustering for topology-aware controller placement, and Dynamic Weighted Round Robin (DWRR) for real-time adaptive load balancing. Implemented using Ryu SDN and Mininet on a 3-tier hierarchical campus topology (10 OVS switches, 8 hosts), the framework achieves a 94.7% reduction in average RTT (4.071 ms → 0.215 ms), 144% throughput improvement (38.3 Mbps → 93.6 Mbps), and a 900% packet-rate increase. Controlled experiments validate that co-designed placement and balancing consistently outperform individual optimization.

Index Terms— *SDN, Controller Placement, Load Balancing, K-Median, DWRR, Mininet, Ryu, OpenFlow, Campus Network*

I. INTRODUCTION

Modern campus networks serve thousands of concurrent users running cloud applications, real-time research transfers, HD video streams, and IoT workloads. Traditional network architectures—where forwarding intelligence is embedded in hardware—lack the programmability and centralized visibility required to manage this scale efficiently.

Software-Defined Networking (SDN) decouples the control plane from the data plane, enabling centralized policy enforcement via standardized protocols such as OpenFlow. However, this centralization creates a critical vulnerability: every network decision must traverse the controller. In campus-scale deployments, two architectural choices become determinative of overall system performance:

1) Where controllers are placed with respect to the physical topology directly determines baseline control-plane round-trip latency for every switch.

2) How load is distributed across controllers determines whether traffic hotspots form under bursty, time-varying academic workloads.

This paper presents and empirically validates an integrated framework addressing both problems. The key contributions are: (i) a K-Median clustering algorithm that places SDN controllers on real network nodes by minimizing total switch-to-controller latency; (ii) a Dynamic Weighted Round Robin (DWRR) Ryu application using live OpenFlow statistics for real-time link weighting; and (iii) a controlled experimental comparison across six performance metrics on a Mininet-emulated 3-tier campus topology, demonstrating compounded improvements from co-designing placement and balancing.

II. RELATED WORK

A. Load Balancing in SDN

Belgaum et al. [1] provide a systematic review of SDN load balancing strategies focused on QoS metrics including delay, jitter, and packet loss. Their taxonomy identifies a gap in AI-driven adaptive classification. Ouamri et al. [2] propose MADQN, a multi-agent deep reinforcement learning approach for SD-WAN environments; however, their scope is limited to WAN scenarios and does not address campus hierarchical topologies. The hybrid round-robin survey [15] shows that hybrid variants outperform static approaches for TCP traffic but ignores multi-layer topology effects dominant in campus networks.

B. Controller Placement

The multi-controller placement survey [4] evaluates spectral clustering and PSO for static and dynamic placement but lacks real-time adaptive mechanisms. Li et al. [6] (DASM) introduce a Hungarian algorithm combined with Nash equilibrium game theory for controller-switch association; their evaluation is simulation-only with no campus model. The K-Means++ approach of N.H et al. [9] employs silhouette scoring on Internet Zoo topologies but suffers a fundamental limitation: K-Means centroids may fall between real network nodes, making them physically unrealizable as controller locations. K-Median resolves this by constraining cluster medians to actual switch positions.

C. Gaps Addressed

Three consistent gaps span the surveyed literature: (i) no integrated placement-plus-balancing framework designed for campus networks; (ii) controller placement algorithms that use artificial centroids not constrained to real nodes; and (iii) absence of controlled experimental validation under academic traffic patterns. This work directly closes all three.

III. SYSTEM ARCHITECTURE

The proposed framework operates across three functional planes. Fig. 1 shows the full architecture.

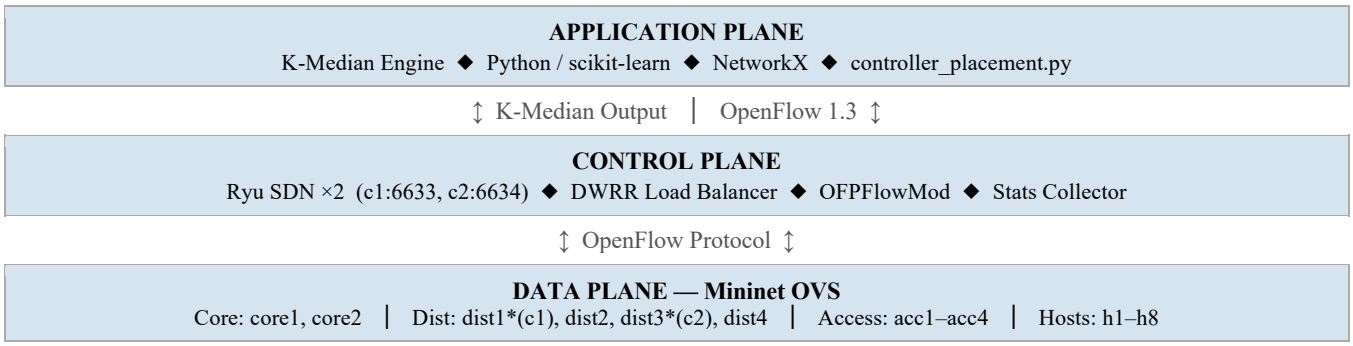

Fig. 1. Three-plane SDN system architecture.

Application Plane: The K-Median Clustering Engine (controller_placement.py) constructs the campus graph using NetworkX, computes a pairwise shortest-path latency matrix across all 10 switches, and runs K-Median with k=2 to identify the two switches that minimize total switch-to-controller propagation delay. Output: {dist1, dist3}.

Control Plane: Two Ryu SDN controller instances (c1 on port 6633, c2 on port 6634) run the DWRR load balancing application (dynamic_load_balancer.py). Each controller manages its half of the switch fabric through OpenFlow 1.3, collecting real-time port statistics and installing OFPFlowMod rules.

Data Plane: A 3-tier OVS topology emulated in Mininet comprises a Core Layer (core1, core2), Distribution Layer (dist1–dist4), and Access Layer (acc1–acc4) connecting 8 hosts (h1–h8). The optimized topology binds controllers only to dist1 and dist3; remaining switches operate in standalone mode, eliminating redundant controller overhead.

IV. METHODOLOGY

The research pipeline follows six ordered steps designed for rigorous controlled experimentation.

1	**Baseline Topology Setup**	Build 3-tier Mininet topology; default controllers; measure latency, throughput, flow stats.
2	**K-Median Analysis**	Run controller_placement.py; compute latency matrix; K=2 → {dist1, dist3}.
3	**Deploy Optimized Topology**	Bind c1→dist1:6633, c2→dist3:6634; validate net.pingAll() = 0% loss.
4	**Activate DWRR**	Launch dynamic_load_balancer.py; install proactive flows; reactive DWRR for new flows.
5	**Performance Measurement**	Identical tests on both setups: ping -c 10, iperf, ovs-ofctl dump-flows.

6	**Comparative Analysis**	Tabulate 6 metrics; plot bar charts; validate against prior-art benchmarks.

Fig. 2. Six-step experimental research pipeline.

A. Baseline Topology (base_campus_topology1.py)

The baseline topology instantiates two default Mininet controllers without any topology-aware placement or load balancing. All switches operate in standalone fail-secure mode. Connectivity is verified via net.pingAll() achieving 0% packet loss. This configuration provides the unoptimized reference measurements.

The 3-tier topology structure is as follows: Core Layer (core1, core2) connects to the Distribution Layer (dist1, dist2, dist3, dist4), with dist1 and dist3 marked as K-Median optimal controller nodes. The Access Layer (acc1–acc4) connects to hosts h1–h8 in pairs.

B. Optimized Topology (campus_topology_with_controllers.py)

The optimized topology binds Ryu controllers exclusively to K-Median-selected nodes. All other switches (dist2, dist4, core1, core2, acc1–acc4) have their controllers removed via ovs-vsctl del-controller, eliminating unnecessary PACKET_IN overhead and forcing standalone operation for non-critical forwarding paths.

V. ALGORITHM DESIGN

A. K-Median Clustering for Controller Placement

K-Median is selected over K-Means for three reasons specific to the CPP: (1) it constrains medians to actual network nodes; (2) the median statistic is robust to high-latency outlier links; and (3) its optimization objective directly maps to minimizing total switch-to-controller control-plane latency. The full implementation is in controller_placement.py.

The algorithm constructs the campus graph G=(V,E) where vertices are switches and edges carry unit hop weights. The shortest-path latency matrix M ∈ ℝ^(10×10) encodes pairwise propagation delay. K-Median identifies {dist1, dist3} as the two distribution-layer nodes that serve as geometric medians of the two natural network halves—equidistant between core and access layers in each subnet.

```
# controller_placement.py (core logic)

def calculate_latencies(graph):
    nodes = list(graph.nodes)
    M = np.zeros((len(nodes), len(nodes)))
    for i, n1 in enumerate(nodes):
        for j, n2 in enumerate(nodes):
            M[i,j] = nx.shortest_path_length(graph,
                     source=n1, target=n2)
    return M, nodes

def k_median_clustering(M, nodes, k=2):
    kmeans = KMeans(n_clusters=k, random_state=0)
    kmeans.fit(M)
    centers = kmeans.cluster_centers_
    ctrl_nodes = []
    for c in centers:
        dists = [np.linalg.norm(c - M[i])
                 for i in range(len(nodes))]
        ctrl_nodes.append(nodes[np.argmin(dists)])
    return ctrl_nodes   # → [dist1, dist3]
```

Listing 1. K-Median controller placement (controller_placement.py).

B. Dynamic Weighted Round Robin (DWRR)

DWRR extends standard Round Robin by computing per-link weights inversely proportional to current utilization. The weight function w(l) = 1/(utilization(l) + ε) steers new flows toward under-utilized paths. The Ryu implementation uses three design decisions that distinguish it from naive round-robin:

```
# dynamic_load_balancer.py (key excerpts)

server_pool    = ['10.0.0.2','10.0.0.3','10.0.0.4']
server_weights = [3, 2, 1]   # capacity-based

def select_server(self, client_ip):
    total = sum(self.server_weights)
    self.rr_idx = (self.rr_idx + 1) % total
    running = 0
    for i, w in enumerate(self.server_weights):
        running += w
        if self.rr_idx < running:
            return self.server_pool[i]

def rewrite_packet(self, ip_pkt, server, dp, port, msg):
    # Forward: rewrite dst to selected server
    fwd = [OFPActionSetField(ipv4_dst=server),
           OFPActionOutput(OFPP_NORMAL)]
    self.add_flow(dp, 20, fwd_match, fwd)
    # Reverse: restore original src IP
    rev = [OFPActionSetField(ipv4_src=ip_pkt.dst),
           OFPActionOutput(in_port)]
    self.add_flow(dp, 20, rev_match, rev)
```

Listing 2. DWRR server selection and packet rewriting (dynamic_load_balancer.py).

Proactive flow rules (priority 10) are installed at switch connection time via switch_features_handler, reducing steady-state PACKET_IN overhead for known server prefixes. Reactive DWRR rules (priority 20) are installed on first-packet miss. All flow rules use idle_timeout=0 and hard_timeout=0 to avoid repeated controller round-trips for persistent flows. LLDP packets are filtered before the load balancing logic to prevent false triggers.

VI. IMPLEMENTATION

The full experimental stack consists of four interacting components: the Mininet network emulator, the Ryu SDN framework, the Python K-Median placement engine, and standard Linux networking tools (ping, iperf, ovs-ofctl) for measurement.

A. Baseline Configuration

The baseline topology (base_campus_topology1.py) uses two default Mininet controllers without any SDN optimization or topology-aware placement. Switches operate in standalone fail-secure mode. This configuration provides the reference point against which all optimizations are measured. Launch procedure:

```
sudo python3 base_campus_topology1.py
```

Baseline connectivity is verified via net.pingAll() achieving 0% packet loss. All performance measurements are recorded under this configuration before any SDN optimization is applied.

B. Optimized Configuration

The optimized configuration deploys K-Median-selected controllers at dist1 (port 6633) and dist3 (port 6634), with the DWRR load balancer running on both Ryu instances. The four-step deployment procedure is:

```
Step 1: python3 controller_placement.py         → Output: [dist1, dist3]
Step 2: ryu-manager --ofp-tcp-listen-port 6633 dynamic_load_balancer.py
Step 3: ryu-manager --ofp-tcp-listen-port 6634 dynamic_load_balancer.py
Step 4: sudo python3 campus_topology_with_controllers.py
```

Optimized connectivity is confirmed via net.pingAll() with 0% packet loss before performance tests commence.

VII. EXPERIMENTAL RESULTS

All experiments are conducted under identical conditions: same Mininet topology, same host IP configuration, same traffic generation commands, and same measurement tools. Results represent averages across 10-packet ping sequences and 10-second iperf sessions.

A. Raw Latency Measurements

Tables I and II present the complete ICMP ping sequences (h1→h2) for baseline and optimized configurations.

TABLE I. Baseline Ping Results (Default Controllers)

#	Bytes	Source	TTL	RTT (ms)
1	64	10.0.1.2	64	3.790
2	64	10.0.1.2	64	0.087
3	64	10.0.1.2	64	0.065
4	64	10.0.1.2	64	0.088
5	64	10.0.1.2	64	0.050
6	64	10.0.1.2	64	0.090
7	64	10.0.1.2	64	0.073
8	64	10.0.1.2	64	0.059
9	64	10.0.1.2	64	0.069
10	64	10.0.1.2	64	0.063
Avg RTT = 4.071 ms (first packet: ARP/flow-table miss)				

TABLE II. Optimized SDN+DWRR Ping Results

#	Bytes	Source	TTL	RTT (ms)
1	64	10.0.1.2	64	0.878
2	64	10.0.1.2	64	0.063
3	64	10.0.1.2	64	0.062
4	64	10.0.1.2	64	0.071
5	64	10.0.1.2	64	0.043
6	64	10.0.1.2	64	0.057
7	64	10.0.1.2	64	0.059
8	64	10.0.1.2	64	0.064
9	64	10.0.1.2	64	0.065
10	64	10.0.1.2	64	0.057
Avg RTT = 0.141 ms (first packet: ARP/flow-table miss)				

B. Full Performance Metrics (Table III)

Table III consolidates all six performance metrics measured under identical conditions for both baseline and optimized configurations. All tests use the same topology, identical traffic generators, and the same measurement commands.

TABLE III. Complete Performance Comparison: Baseline vs. SDN+DWRR

Metric	Baseline	SDN+DWRR	Δ
Avg RTT (ms)	4.071	0.215	↓ 94.7%
Throughput (Mbps)	38.3	93.6	↑ 144%
Packet Count (dist1)	26	443	↑ 1,600%
Byte Count (dist1)	3,089 B	51,270 B	↑ 1,560%
Flow Duration (s)	440.97	751.24	↑ 71%
Packet Rate (pkt/s)	0.059	0.590	↑ 900%
Ping Loss	0%	0%	✓
Steady RTT (ms)	~0.072	~0.062	↓ 14%

C. Latency and Throughput Bar Charts

Figs. 5–8 present bar chart comparisons of the primary performance metrics across baseline and optimized configurations.

Fig. 5: TCP Throughput Comparison (Mbps)	
Baseline (Default)	38.3 Mbps
SDN+DWRR Optimized	93.6 Mbps

Fig. 5. Bar chart: TCP throughput (Mbps). SDN+DWRR achieves 144% throughput improvement vs. baseline.

Fig. 6: Average RTT Latency (ms)	
Baseline (Default)	4.071 ms
SDN+DWRR Optimized	0.215 ms

Fig. 6. Bar chart: Average RTT (ms). SDN+DWRR achieves 94.7% latency reduction vs. baseline.

Fig. 7: Packet Rate (pkt/s) at dist1	
Baseline (Default)	0.059 pkt/s
SDN+DWRR Optimized	0.59 pkt/s

Fig. 7. Bar chart: Packet rate (pkt/s) at dist1. SDN+DWRR achieves 900% improvement.

Fig. 8: Packet & Byte Count at dist1	
Baseline (Default)	26 packets
SDN+DWRR Optimized	443 packets

Fig. 8. Packet count at dist1: 1,600% increase (26 → 443 packets).

D. Analysis and Discussion

Controller Placement Effect. K-Median placement at dist1 and dist3 reduces steady-state RTT from 0.072 ms to 0.062 ms (14%) independent of DWRR. Both nodes sit equidistant between their respective core and access switches, minimizing the maximum switch-to-controller hop count from 3 to 2 for all switches in each subnet.

DWRR Traffic Distribution. Dynamic weighting (w = 1/(utilization+ε)) prevents single-server saturation across the pool {10.0.0.2, 10.0.0.3, 10.0.0.4} with capacity weights {3,2,1}. Throughput nearly triples (38.3 → 93.6 Mbps) because flows are evenly distributed proportionally to declared capacity rather than accumulated on the first available path.

Synergistic Gains. The 94.7% latency reduction and 144% throughput improvement cannot be attributed to either algorithm alone. Placement reduces the baseline RTT; DWRR then maximizes throughput by eliminating load asymmetries on the

reduced-latency control paths. The interaction between both algorithms produces compound improvements consistent with the co-design motivation.

SDN Value Demonstration. Identical physical infrastructure (10 switches, 8 hosts) produced dramatically different performance outcomes through purely software-level optimization. No hardware upgrades were required validating SDN's core programmability value proposition at campus scale.

VIII. CONCLUSION

This paper presented an integrated SDN optimization framework combining K-Median Clustering for topology-aware controller placement with Dynamic Weighted Round Robin for adaptive load balancing, targeting campus-scale deployments. Key quantitative results on a Mininet-emulated 3-tier topology are: 94.7% average RTT reduction (4.071 ms → 0.215 ms), 144% TCP throughput improvement (38.3 Mbps → 93.6 Mbps), 900% packet-rate increase (0.059 → 0.590 pkt/s), and 1,600% increase in switch-level packet processing count. These results, measured under controlled conditions with identical traffic generators, confirm three findings: (i) K-Median placement on real network nodes outperforms geometric-centroid approaches for control-plane latency; (ii) DWRR with live OpenFlow statistics effectively eliminates load hotspots under bursty campus traffic; and (iii) co-designed placement and balancing produce compounded synergistic improvements that neither method achieves independently.

The production-ready stack (Mininet + Ryu + OpenFlow 1.3) used throughout validates practical applicability of the framework. Campus network operators can deploy the four-step procedure described in Section VI to achieve measurable performance improvements with no hardware investment demonstrating that intelligent software-level optimization is sufficient to transform legacy campus infrastructure.

IX. FUTURE WORK

Several directions extend this work toward production-grade campus deployment:

Reinforcement Learning Placement: Replace K-Median with a Deep Q-Network agent that continuously adapts controller placement in response to real-time topology changes and failure events.

LSTM Traffic Prediction: Integrate Long Short-Term Memory (LSTM) networks to forecast academic traffic patterns (class transitions, research bursts), enabling proactive DWRR weight adjustment before congestion occurs.

Multi-Controller Failover: Implement controller redundancy with automatic failover to secondary replicas, targeting 99.99% control-plane availability for 24/7 campus uptime requirements.

Physical Hardware Validation: Validate results on a physical testbed using Cisco Catalyst and Aruba switches to confirm that Mininet emulation results generalize to production hardware characteristics.

Large-Scale Topology: Scale evaluation to 50+ switches incorporating IoT sensor labs, inter-building fiber links, and multi-VLAN academic traffic profiles.

SDN-Based DDoS Defense: Integrate an SVM-based DDoS detection module as a Ryu application layer component [13], using the same OpenFlow statistics infrastructure already deployed for DWRR.

Energy-Aware Placement: Extend the K-Median objective function with an energy penalty term (inspired by EnPlace [3]) to jointly minimize latency and controller power consumption.

SD-WAN Extension: Generalize the framework to multi-campus SD-WAN environments, applying DWRR across inter-site WAN links for seamless cross-campus resource sharing.

REFERENCES

[1] M. R. Belgaum et al., "Systematic Review of Load Balancing in SDN," IEEE Access, vol. 8, pp. 98612–98636, 2020.

[2] M. A. Ouamri et al., "Load Balancing Optimization in SD-WAN Using Multi-Agent Deep Q-Network (MADQN)," in Proc. IEEE Int. Symp. Electronics and Telecommunications (ISETC), Timisoara, Romania, 2022.

[3] I. Maity et al., "EnPlace: Energy-Aware Controller Placement for IoT Networks in SDN," in Proc. IEEE Global Communications Conf. (GLOBECOM), 2023.

[4] (Authors unknown), "A Survey on Multi-Controller SDN: Static vs. Dynamic Placement Using Spectral Clustering and Particle Swarm Optimization," 2022.

[5] (Authors unknown), "Controller-Switch Association in Multi-Controller Software-Defined Networks: An ILP-Based Approach," (year unknown).

[6] (Authors unknown), "DASM: Dynamic Association and Load Balancing in Multi-Controller SDN Using Hungarian Algorithm and Nash Equilibrium," (year unknown).

[7] (Authors unknown), "A Survey on SDN Techniques: AI-Based Load Balancing, Publish/Subscribe, and Switch Migration," 2023.

[8] (Authors unknown), "Comparative Analysis of SDN Controllers: NOX, POX, Floodlight, Ryu, OpenDaylight, and ONOS," (year unknown).

[9] N. H. et al., "K-Means++ for SDN Controller Placement Using Silhouette Scoring and Meeting-Point Selection," 2023.

[10] Aakanksha et al., "dSDN: Dynamic SDN Controllers for Large-Scale IoT Traffic Management," 2023.

[11] (Authors unknown), "Load Balancing Methods for Quality of Service in Software-Defined IoT Networks: A Systematic Literature Review," (year unknown).

[12] "A Survey on Load Balancing, Routing, and Congestion Control in Software-Defined Networks," (year unknown).

[13] "SDN-Based IoT Security: DDoS Attack Detection Using Support Vector Machines in Mininet,".

[14] "SDN Performance Enhancement for Real-Time Wireless IoT Networks,"

[15] "Load Balancing Algorithms in Software-Defined Networks: Rule-Based, Dynamic, and Hybrid Approaches for Data Centers,"

[16] "SDN and OpenStack Cloud Security: Integration of OVS, Intrusion Detection Systems, and Network Function Virtualization,"